UBM Tech

Tech Digest

InformationWeek

JUNE 2014

Next

DOWNLOAD PDF

Using C++ and COM with WinRT

Powered by Dr.Dobb's

PLUS: Getting the most from the Microsoft C++ REST SDK >>

Using C++ and COM with WinRT

Working with C++ in the world of Windows Store Apps

by Christophe Pichaud

Traditional Windows C++ development is powered by Win32 API calls. With Windows 8 platforms, however, applications must call a set of components named the Windows Runtime APIs. These components are also referred to as the WinRT API. This API is available for various languages: C#, VB, JavaScript, and C++. For C++ developers, there are two options: C++/CX or standard ISO C++. The first option, C++/CX, is a set of compiler extensions with the existing syntax of the C++/CLI plus ^ and % characters. C++/CX generates native code and hides the plumbing of WinRT. C++/CX is very well documented with samples. The other option is to choose standard C++ programming with a new library: Windows Runtime Library (WRL). This is the option Microsoft uses to develop WinRT APIs.

Windows Runtime Library

WRL is the successor of the popular Active Template Library (ATL), which allows developers to simplify the programming of COM components. WRL is inspired by ATL: It provides C++ templates and internally uses meta-programming and modern C++ techniques to easily develop new COM components. The source code of WRL is shipped as C++ headers with the Windows SDK. WRL is documented in MSDN (http://is.gd/Ywd3tc), but there are few samples.

Enhancements to the COM Technology

Windows 8 brings new features to the COM technology with the new `IInspectable` interface that defines three new methods: `GetIids`, `GetRuntimeClassName`, and `GetTrustLevel`. The Windows Runtime is powered by COM, but it does not embrace the OLE technology. Everything relies on `IInspectable`, which inherits from `IUnknown`. There is no support for the `IDispatch` interface, ActiveX controls, or connection points. Importantly, the `VARIANT` data-type is not supported in Windows Store Apps.

But there is a new Windows string type named `HSTRING`. There are also a lot of new interfaces that cover various aspects like asynchronous operations, collections containers, and iterators that are exposed for all the languages. For the VB and C# languages, asynchronous operations are handled by `async` and `await` keywords. Collections are mapped to .NET framework interfaces like `IList<T>` or `IDictionary<K, V>`. The `HSTRING` data-type is mapped to `System.String` for .NET languages.

For C++, there is no projection layer, so the practice is just to make standard COM programming tasks with WRL and Windows types. COM programming requires interface and coclass elements to be defined in an IDL file. The MIDL compiler produces C source code for the proxy/stub and a type library (TLB). From there, you follow the same C++ conventions with minor changes. Your interfaces inherit from `IInspectable` and your `runtimeclass` elements (the old term was `coclass`) contain one or more interfaces. The new MIDL compiler produces the C proxy/stub and a Windows metadata (WINMD) file. This winmd file can be incorporated as a reference to C#, VB, or HTML5/JS projects.

Writing a Simple WinRT Component

To start WRL programming, you can download the WRL project template (http://is.gd/QjvIkn). This project template creates a basic WRL library project with an IDL file, a simple component, and a module that exposes the plumbing around the class factory. There is no wizard to add properties or methods to your class. You have to modify the IDL and the class source code files (.h and .cpp files) as in traditional COM programming.

```
// Library1.IDL
import "inspectable.idl";
import "Windows.Foundation.idl";

#define COMPONENT_VERSION 1.0
namespace Library1
{
    interface ILogger;
    runtimeclass Logger;

    [uuid(3EC4B4D6-14A6-4D0D-BB96-31DA25224A15),
        version(COMPONENT_VERSION)]
    interface ILogger : IInspectable
    {
        [propget] HRESULT Name([out, retval] HSTRING* value);
        [propput] HRESULT Name([in] HSTRING value);
        HRESULT LogInfo([in] HSTRING value);
```

```
    }

    [version(COMPONENT_VERSION), activatable(COMPONENT_VERSION)]
    runtimeclass Logger
    {
        [default] interface ILogger;
    }
}
```

If you want to use standard COM programming, you have to provide your own infrastructure to embrace the WinRT interfaces with STL containers.

The class is declared in the ABI namespace. This is a required convention from Microsoft. The class inherits from a `RuntimeClass<T>` template that takes our interface as a template parameter. There are two macros in this source code:

- `InspectableClass` defines the name of the WinRT component and its trust level.
- `ActivatableClass` defines the class factory plumbing for that class.

Each method has to return `HRESULT`, so you can reuse the `COM_STDMETHOD()` macro that is expanded as `virtual __declspec(nothrow) HRESULT __stdcall`.

Real-World Scenario: Returning a Collection of Objects

When you write real-world COM components, you have to return data and you need more than a string or a basic type. The best practice is to return objects in a collection that can be used with standard conventions like supporting the `ForEach` iteration pattern in other languages.

In standard COM/OLE, you have to implement `Count` and `Item` properties to support Visual Basic `ForEach`. With the Windows Runtime, you have to follow the definition of some interfaces used by language projections to enable a client application in C#, for example, to do:

```
// C# client
Library1.Root root = new Library1.Root();
IList<String> list = root.GetVector();
foreach(string s in list)
   { ... }
```

At this point, you enter into undocumented features of the Windows Runtime. To find the appropriate information, you need to watch the Channel9 video and open the PowerPoint material (http://channel9.msdn.com/Events/BUILD/BUILD2011/PLAT-874T) from the Windows Runtime session presented at the BUILD 2011 conference (that covered the preview of Windows 8). It covers some interfaces named `IVector<T>`, `IVectorView<T>`, and `IMap<T>`. Using Visual Studio 2012 or Visual Studio 2013, you will discover that theses interfaces are defined in windows.foundation.collections.h. This file is included in the Windows SDK's include\winrt folder. If you look at the Microsoft BUILD conference materials, you will gather that there is a deep integration with STL. With C++/CX, a header called collection.h (described as Windows Runtime Collection/Iterator Wrappers) provides the necessary plumbing between standard STL containers and WinRT interfaces. But

if you want to use standard COM programming, you have to provide your own infrastructure to embrace the WinRT interfaces with STL containers. So the deep STL integration is really only for C++/CX. For raw COM programming using standard ISO C++, we need to implement the various interfaces within C++ classes.

Reading the windows.foundation.collections.h file and looking at MSDN information on the `IVector<T>` interface (http://is.gd/bCEKqq), you'll notice `IVector<T>` inherits from `IIterable<T>`, which in turn introduces `IIterator<T>` in the `First()` method. So, we have to implement these interfaces and their methods into a dedicated component. A new feature appears in the definition of these interfaces. They are defined like generics or templates. Let's examine these interfaces (I have made a little modification):

```
template <class T>
class IVector : IInspectable /* requires IIterable<T> */
{
public:
    // read methods
    virtual HRESULT GetAt(unsigned index, T *item) = 0;
    virtual HRESULT get_Size(unsigned *size) = 0;
    virtual HRESULT GetView(IVectorView<T> **view) = 0;
    virtual HRESULT IndexOf(T value, unsigned *index,
        boolean *found) = 0;
    // write methods
    virtual HRESULT SetAt(unsigned index, T item) = 0;
    virtual HRESULT InsertAt(unsigned index, T item) = 0;
    virtual HRESULT RemoveAt(unsigned index) = 0;
    virtual HRESULT Append(T item) = 0;
    virtual HRESULT RemoveAtEnd() = 0;
    virtual HRESULT Clear() = 0;
    // bulk transfer methods
    virtual HRESULT GetMany(unsigned startIndex,
        unsigned capacity, T *value, unsigned *actual) = 0;
    virtual HRESULT ReplaceAll(unsigned count, T *value) = 0;
};

template <class T>
class IIterable : IInspectable
{
public:
    virtual HRESULT First(IIterator<T> **first) = 0;
};

template <class T>
class IIterator : IInspectable
{
public:
    virtual HRESULT get_Current(T *current) = 0;
    virtual HRESULT get_HasCurrent(boolean *hasCurrent) = 0;
    virtual HRESULT MoveNext(boolean *hasCurrent) = 0;
    virtual HRESULT GetMany(unsigned capacity, T *value,
        unsigned *actual) = 0;
};
```

The `IVector<T>` interface contains all the access methods required for a vector. It is not difficult to associate these methods with the `std::vector<T>` methods. The `IIterable<T>` interface allows returning an iterator via `IIterator<T>`. The `Iterator<T>` interface represents the same concept of iterator that we know in the STL.

To implement the necessary plumbing, I will create two template classes. The first class `Vector<T>` will implement `IVector<T>` and `IIterable<T>` using a `std::vector<T>`. The second class `Iterator<T>` will implement `IIterator<T>` using the `std::vector<T>::iterator` iterator. Theses template classes are exposed as WinRT classes by inheriting from `RuntimeClass<T>`. Here is the source code:

```
template <class T>
class Iterator : public RuntimeClass<IIterator<T>>
{
    InspectableClass(L"Library1.Iterator", BaseTrust)

private:
    typedef typename std::shared_ptr<std::vector<T>> V;
    typedef typename std::vector<T>::iterator IT;

public:
    Iterator() {}

    HRESULT Init(const V & item)
    {
        _v = item;
        _it = _v->begin();
        if (_it != _v->end())
        {
            _element = *_it;
            _bElement = TRUE;
        }
        else
            _bElement = FALSE;
        return S_OK;
    }

public:
    virtual HRESULT STDMETHODCALLTYPE get_Current(T *current)
    {
        if (_it != _v->end())
        {
            _bElement = TRUE;
            _element = *_it;
            *current = _element;
        }
        else
        {
            _bElement = FALSE;
        }
        return S_OK;
    }

    virtual HRESULT STDMETHODCALLTYPE get_HasCurrent(boolean
        *hasCurrent)
    {
        if (_bElement)
            *hasCurrent = TRUE;
        else
            *hasCurrent = FALSE;
        return S_OK;
    }

    virtual HRESULT STDMETHODCALLTYPE MoveNext(boolean
        *hasCurrent)
    {
        if (_it != _v->end())
        {
            ++_it;
            if (_it == _v->end())
                *hasCurrent = FALSE;
            else
                *hasCurrent = TRUE;
        }
        else
        {
            *hasCurrent = FALSE;
        }
        return S_OK;
    }

private:
    V _v;
    IT _it;
    T _element;
```

```
    boolean _bElement = FALSE;
};

template <typename T>
class Vector : public RuntimeClass<IVector<T>, IIterable<T>>
{
    InspectableClass(L"Library1.Vector", BaseTrust)

public:
    Vector()
    {
        std::shared_ptr<std::vector<T>> ptr(new
std::vector<T>());
        _v = ptr;
    }

public:

    // read methods
    virtual HRESULT STDMETHODCALLTYPE GetAt(unsigned index,
        T *item)
    {
        *item = (*_v)[index];
        return S_OK;
    }

    virtual HRESULT STDMETHODCALLTYPE get_Size(unsigned *size)
    {
        *size = _v->size();
        return S_OK;
    }

    virtual HRESULT STDMETHODCALLTYPE GetView(IVectorView<T>
        **view)
    {
        return E_NOTIMPL;
    }

virtual HRESULT STDMETHODCALLTYPE IndexOf(T value,
    unsigned *index, boolean *found)
{
    unsigned foundedIndex = 0;
    bool founded = false;
    for (std::vector<T>::const_iterator it =
        _v->cbegin() ; it != _v->end() ; ++it)
    {
        const T & v = *it;
        if (v == value)
        {
            founded = true;
            break;
        }
        foundedIndex++;
    }
    if (founded == true)
    {
        *found = TRUE;
        *index = foundedIndex;
    }
    else
    {
        *found = FALSE;
    }
    return S_OK;
}

// write methods
virtual HRESULT STDMETHODCALLTYPE SetAt(unsigned index,
    T item)
{
    if (index < _v->size())
    {
        (*_v)[index] = item;
    }
    return S_OK;
}
```

```
virtual HRESULT STDMETHODCALLTYPE InsertAt(unsigned index,
    T item)
{
    _v->emplace(_v->begin() + index, item);
    return S_OK;
}

virtual HRESULT STDMETHODCALLTYPE RemoveAt(unsigned index)
{
    if (index < _v->size())
    {
        _v->erase(_v->begin() + index);
    }
    return S_OK;
}

virtual HRESULT STDMETHODCALLTYPE Append(T item)
{
    _v->emplace_back(item);
    return S_OK;
}

virtual HRESULT STDMETHODCALLTYPE RemoveAtEnd()
{
    if (!_v->empty())
    {
        _v->pop_back();
    }
    return S_OK;
}

virtual HRESULT STDMETHODCALLTYPE Clear()
{
    _v->clear();
    return S_OK;
}
    virtual HRESULT STDMETHODCALLTYPE First(IIterator<T>
        **first)
    {
        ComPtr<Iterator<T>> p = Make<Iterator<T>>();
        p->Init(_v);
        *first = p.Detach();
        return S_OK;
    }

private:
    std::shared_ptr<std::vector<T>> _v;
};
```

You will notice that access to the `std::vector<T>` can be enhanced and better protected.

Now that we have the plumbing to deal with collections, let's use it in a WinRT component with two methods. The first method returns a vector of strings, and the second method returns a vector of integers. Here is the IDL file:

```
   [uuid(3EC4B4D6-14A6-4D0D-BB96-31DA25224A16),
version(COMPONENT_VERSION)]
    interface IRoot : IInspectable
    {
        HRESULT GetVector([out][retval] Windows.Foundation.
            Collections.IVector<HSTRING>** value);
        HRESULT GetVectorInt([out][retval] Windows.Foundation.
            Collections.IVector<int>** value);
    }

    [version(COMPONENT_VERSION),
        activatable(COMPONENT_VERSION)]
    runtimeclass Root
    {
        [default] interface IRoot;
```

```
    }
```

Here is the C++ class header that implements the `IRoot` interface:

```
namespace ABI
{
    namespace Library1
    {
        class Root : public RuntimeClass<IRoot>
        {
            InspectableClass(L"Library1.Root", BaseTrust)

        public:
            Root() {}

        public:
            STDMETHOD(GetVector)(IVector<HSTRING>** value);
            STDMETHOD(GetVectorInt)(IVector<int>** value);
        };

        ActivatableClass(Root);
    }
}
```

Here are the two methods of the `Root` C++ class:

```
namespace ABI
{
    namespace Library1
    {
        STDMETHODIMP Root::GetVector(IVector<HSTRING>** value)
        {
            ComPtr<Vector<HSTRING>> v =
                Make<Vector<HSTRING>>();
            HString str;
            str.Set(L"String1");
            v->Append(str.Detach());
            str.Set(L"String2");
            v->Append(str.Detach());
            str.Set(L"String3");
            v->Append(str.Detach());
            str.Set(L"String4");
            v->Append(str.Detach());
            *value = v.Detach();
            return S_OK;
        }

        STDMETHODIMP Root::GetVectorInt(IVector<int>** value)
        {
            ComPtr<Vector<int>> v = Make<Vector<int>>();
            v->Append(10);
            v->Append(20);
            v->Append(30);
            v->Append(40);
            *value = v.Detach();
            return S_OK;
        }
    }
```

The `Vector<T>` template class is used in both methods. To create a WinRT class, you need to wrap your component in a `ComPtr<T>` object and use the `Make<T>` function. You can also see the use of the `HString` class to handle the new `HSTRING` Windows type.

The complete source code for this article is available as a working sample (http://is.gd/d1kr6G) in the Microsoft Dev Center: Windows Store apps.

Christophe Pichaud is a software engineer from Paris, France. He works for Sogeti (http://www.sogeti.com) and uses C++ and C#.

Comment

Using the Microsoft C++ REST SDK

The new SDK enables you to stay in C++ when consuming REST services

By Gastón Hillar

Visual Studio 2013 includes the C++ REST SDK version 1.0, code-named Casablanca. This Microsoft open source project is evolving in CodePlex (http://casablanca.codeplex.com/), and takes advantage of the new set of capabilities introduced in C++ 11 to simplify cloud-based coding with a modern, asynchronous, and multi-platform API design. In this article, I explain how you can use the C++ REST SDK to consume REST services.

Understanding C++ REST SDK Architecture

When you need the best performance, you usually evaluate going native, and C++ is one of the best options for doing so. Microsoft believes C++ is valuable in the cloud; and the company's new C++ REST SDK enables developers to work with C++ to consume REST services and achieve both great performance and scalability. It allows you to stay in C++ when consuming REST services or developing other code closely related to the cloud.

If you use C++ to consume cloud services but you use a C-based and synchronous API with callbacks, you aren't taking full advantage of the improvements included in the latest C++ versions. In addition, your code will be difficult to read and debug, and the synchronous API will make it difficult for you to create a responsive UI. Most modern Web APIs try to reduce unnecessary boilerplate and so offer asynchronous methods without the complexity of C-style callbacks.

For example, if you work with C++ 11 but you use it to make calls to a synchronous C-based API to make an HTTP `GET` call, your productivity levels cannot even be compared to other programming languages such as C# or Python. Microsoft developed the C++ REST SDK on top of the Parallel Patterns Library (PPL), to leverage PPL's task-based programming model. Whenever you perform an asynchronous operation with the C++ REST SDK, you are creating a new PPL task. To make the C++ REST SDK portable to Linux, Microsoft made the necessary portions of PPL run on Linux (and compile cleanly with GCC). Thus, the C++ REST SDK uses a concurrency runtime for C++ that relies heavily on C++ 11 features. Instead of working with callbacks, you can write elegant C++ 11 code that creates tasks and schedules other tasks to be executed when certain tasks finish execution. If you have previous experience with PPL, you will find it easier to work with the C++ REST SDK.

The C++ REST SDK relies on the following four low-level stacks or APIs that ride on top of the services provided by the different operating system (see Figure 1):

- **WinHTTP:** also known as Microsoft Windows HTTP Services. It is a C-based HTTP client API.
- **PPL** (short for Parallel Patterns Library): the programming model for composing asynchronous operations. The C++ REST SDK uses WinHTTP on different Windows versions.
- **Boost.Asio:** a cross-platform C++ library for network and low-level I/O programming that provides a consistent asynchronous model. The library uses a modern C++ approach. The C++ REST SDK uses Boost.Asio to manage communications on Linux.
- **HTTP.sys**: the Windows server-side API for HTTP. The C++ REST SDK uses HTTP.sys on different Windows versions.
-

At the time of writing, the C++ REST SDK supports the following operating systems. Note that support for both Windows XP and Windows Phone 8.x is still considered experimental.

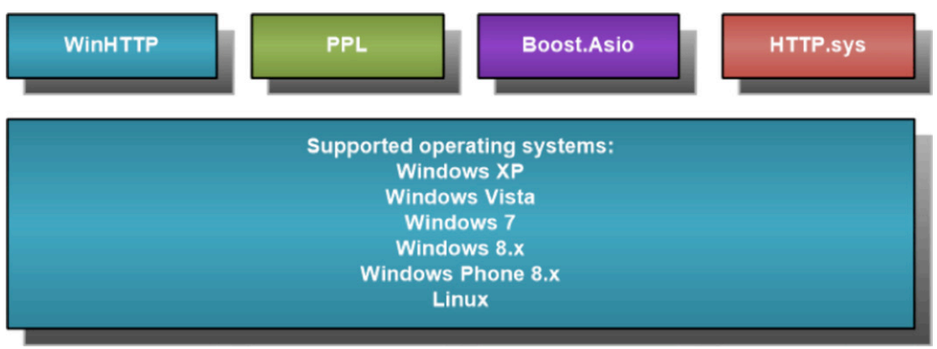

Figure 1: The four low-level stacks used by the C++ REST SDK.

- Windows XP
- Windows Vista
- Windows 7
- Windows 8.x
- Windows Phone 8.x
- Linux

The C++ REST SDK offers the following pieces that use the different low-level stacks (see Figure 2):

- **Asynchronous streams and stream buffers:** These provide a base asynchronous stream, a base asynchronous stream buffer, and many implementations, such as asynchronous file streams. There are specific Interop Streams that support interoperability between STL iostreams and C++ REST SDK asynchronous streams.
- **HTTP client and listener:** The HTTP client allows you to maintain a connection to an HTTP service and send requests to the server. The HTTP listener allows you to accept messages directed at a particular URI.
- **JSON parser and writer:** JSON is usually a problem for static-type languages such as C++. The C++ REST SDK uses a single class `(web::json::value)` to represent JSON values and provides the necessary helpers to assist in serialization. You can determine the type associated with this JSON value at runtime. The C++ REST SDK enables you to pull JSON values from streams by using the parser and write JSON values to streams.
- **TCP client and listener:** The TCP client provides client connections for TCP network services. The TCP listener listens for connections from TCP network clients. The primary benefit of the

TCP client and listener is that it works with non-blocking asynchronous operations. However, these components are still considered experimental. If you have ever worked with the `System.Net.Sockets::TcpListener` and `System.Net.Sockets::TcpClient` classes, you will enjoy working with the experimental TCP client and listener and its simpler non-blocking asynchronous operations.

The Microsoft team working with the C++ REST SDK has plans for adding these pieces in the near future:

- Web Sockets client and listener.
- UDP client and listener.
- Additional product-specific APIs built on top of the C++ REST SDK, such as Azure Storage, Mobile Services, and Bing Maps.

Working with the C++ REST SDK in a Visual Studio Project

To create a project that uses the SDK in Visual Studio and builds without unexpected issues, you need to download and install the latest version from CodePlex at http://casablanca.codeplex.com/. I use version 1.3 in my examples, so if you download a different version, you will have to change some configuration values.

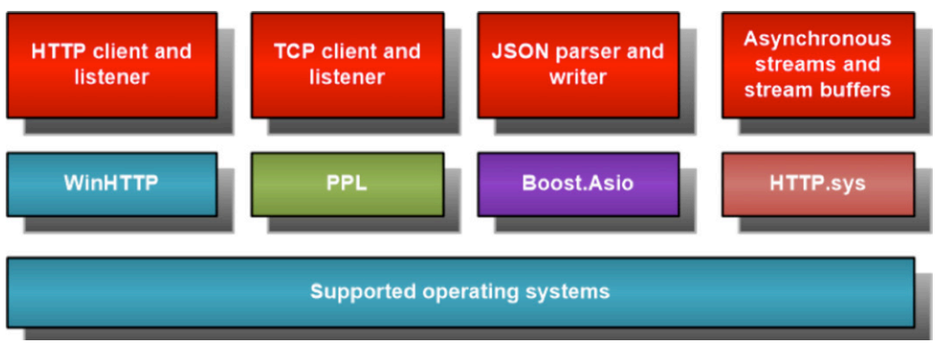

Figure 2: Pieces that use the different low-level stacks.

The default installation folder for version 1.3 is `C:\Program Files (x86)\Microsoft Cpp REST SDK for VS 2013 v1.3\SDK`. The `VSProject` subfolder includes two property files for your Visual Studio projects: `CppRest.props` and `version.props`.

After you create a C++ project for which you want to use the C++ REST SDK, you must follow these steps:

- Copy the two property files found in the `VSProject` subfolder (`CppRest.props` and `version.props`) within the `C++ REST SDK` installation folder and paste them in the main folder in which you created your Visual Studio project.
- Open the `cpprest.props` file where there are two properties that might have wrong values: `CppRestSDKPath` and `CppRestRuntimeBaseFileName`. If you find the following two lines, you will need to replace the 110 value with 120, as shown in the next lines. In 64-bit Windows versions, the key value name that defines the base installation folder for the C++ REST SDK is `InstallDir` within `HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Microsoft\CppRestSDK\OpenSourceRelease\120\v1.3\SDK`. Unluckily, some versions of the SDK didn't check the property values and you won't be able to have a successful build if you don't replace 110 with 120 because the registry entry will never be found during the build process. The samples included with the C++ REST SDK also have the same problem, and therefore, you must make the replacements in the property files to build them.

The lines with wrong values (to be replaced) are:

```
<CppRestSDKPath>$([MSBuild]::GetRegistryValue(`HKEY_LOCAL_MA-
CHINE\Software\Microsoft\CppRestSDK\OpenSourceRelease\110\
v$(CppRestSDKVersionString)\SDK`, `InstallDir`))</CppRestSDK-
Path>
<CppRestRuntimeBaseFileName>$(CppRestBaseFileName)110$(DebugFi
leSuffix)_$(CppRestSDKVersionFileSuffix)</CppRestRuntimeBase-
FileName>
```

The lines with the right values:

```
<CppRestSDKPath>$([MSBuild]::GetRegistryValue(`HKEY_LOCAL_MA-
CHINE\Software\Microsoft\CppRestSDK\OpenSourceRelease\120\
v$(CppRestSDKVersionString)\SDK`, `InstallDir`))</CppRestSDK-
Path>
<CppRestRuntimeBaseFileName>$(CppRestBaseFileName)120$(DebugFi
leSuffix)_$(CppRestSDKVersionFileSuffix)</CppRestRuntimeBase-
FileName>
```

- Now open the Visual Studio `vcxproj` file for your C++ project in your favorite editor and add the following line within the main `Project` element. This way, the project will include the `CppRest.props` definition. Notice that `CppRest.props` includes a line that imports the previously copied `version.props` file: `<Import Project="CppRest.props" />`
- Open the project in Visual Studio and go to the project's properties. Add the entire path to the include a subfolder of the C++ REST SDK installation folder to the `Include Directories` within `Configuration Properties | VC++ Directories`. If you used the default installation folder, you need to add `C:\Program Files (x86)\Microsoft Cpp REST SDK for VS 2013 v1.3\SDK\include`.
- Add the entire path to the `lib` subfolder of the C++ REST SDK installation folder to the `Library Directories` within `Configuration Properties | VC++ Directories`. If you used the default installation folder, you need to add `C:\Program Files (x86)\Microsoft Cpp REST SDK for VS 2013 v1.3\SDK\lib`.
- Add the C++ REST SDK library, `cpprest120d_1_3.lib`, to Additional Dependencies within `Configuration Properties | Linker | Input`.

Now your C++ project is ready to start working with the C++ REST SDK. (You could also install the latest C++ REST SDK version by running the command `Install-Package cpprestsdk` in the Package Manager Console, and using the package to add the C++ REST SDK to an existing project with the properties set for you. However, the best way to make the latest versions work with a new project without problems is to perform the steps I've outlined.)

Executing an Asynchronous HTTP GET Call with Parameters

The following code shows an example of a Windows console application that calls Flickr via the C++ REST SDK, using an HTTP `GET` call:

```
http://api.flickr.com/services/rest/?method=flickr.test.
echo&name=value
```

and retrieve the status code returned by the server, the response headers type and length, and the response body. (Before using the code, you need to follow the steps explained in the previous section for your Visual Studio project.)

```
// The code includes the most frequently used includes
// necessary to work with C++ REST SDK
#include "cpprest/containerstream.h"
#include "cpprest/filestream.h"
#include "cpprest/http_client.h"
#include "cpprest/json.h"
#include "cpprest/producerconsumerstream.h"
#include <iostream>
#include <sstream>

using namespace ::pplx;
using namespace utility;
using namespace concurrency::streams;

using namespace web::http;
using namespace web::http::client;
using namespace web::json;

pplx::task<void> HTTPGetAsync()
{
    // I want to make the following HTTP GET:
    //  http://api.flickr.com/services/rest/?method=flickr.
    //  test.echo&name=value
http_client client(U("http://api.flickr.com/services/rest/"));

    uri_builder builder;
    // Append the query parameters:
    // ?method=flickr.test.echo&name=value
    builder.append_query(U("method"), U("flickr.test.echo"));
    builder.append_query(U("name"), U("value"));

    auto path_query_fragment = builder.to_string();

    // Make an HTTP GET request and asynchronously process
    //  the response
    return client.request(methods::GET, path_query_fragment).
        then([](http_response response)
    {
        // Display the status code that the server returned
        std::wostringstream stream;
        stream << L"Server returned returned status code "
            << response.status_code() << L'.' << std::endl;
        std::wcout << stream.str();

        stream.str(std::wstring());
        stream << L"Content type: " << response.headers().
            content_type() << std::endl;
        stream << L"Content length: " << response.headers().
            content_length() << L"bytes" << std::endl;
        std::wcout << stream.str();

        auto bodyStream = response.body();
        streams::stringstreambuf sbuffer;
        auto& target = sbuffer.collection();

        bodyStream.read_to_end(sbuffer).get();

        stream.str(std::wstring());
        stream << L"Response body: " << target.c_str();
        std::wcout << stream.str();
    });
}

#ifdef _MS_WINDOWS
int wmain(int argc, wchar_t *args[])
#else
int main(int argc, char *args[])
#endif
{
    std::wcout << L"Calling HTTPGetAsync..." << std::endl;
    // In this case, I call wait. However, you won't usually
    // want to wait for the asynchronous operations
    HTTPGetAsync().wait();

    return 0;
}
#ifdef _MS_WINDOWS
int wmain(int argc, wchar_t *args[])
#else
int main(int argc, char *args[])
#endif
{
    std::wcout << L"Calling HTTPGetAsync..." << std::endl;
       // In this case, I call wait. However, you won't usually
```

```
    // want to wait for the asynchronous operations
    HTTPGetAsync().wait();

    return 0;
}
```

The `wmain` method in Windows calls the `HTTPGetAsync` method that returns a `pplx::task<void>` instance that represents an asynchronous operation. In this case, the code calls the `wait` method for the returned task instance because the only goal for this sample console application is to display the results. However, you usually won't want to call `wait` for the returned task instance, so that you don't block the UI.

The `HTTPGetAsync` method creates a new instance of the `web::http::client:http_client` class to maintain a connection with the `Flickr REST API` whose base URI is `http://api.flickr.com/services/rest/`. The C++ REST SDK works with platform-independent strings, so in order to deal with string literals in a platform-specific way, the code uses the `U(str)` macro, which takes a string literal and produces the platform-specific type. The `typedef utility::string_t` corresponds to the platform preference.

```
http_client client(U("http://api.flickr.com/
services/rest/"));
```

At this point, the code has the client and it is necessary to determine the relative URI that the request should use. The path is `/services/rest` and it is already included in the URI used to create the HTTP client. Therefore, there is no need to specify it. The code creates an instance of the `web::http::uri::builder` class that allows you to build URIs incrementally. It is a convenient way to append the query fragments in an elegant way (by using the `U(str)` macro). The two calls to the `append_query` method add the necessary queries to the URI builder, encoding them first. Then, the call to the `to_string` method for the builder generates the necessary query fragment to make the request.

```
uri_builder builder;
builder.append_query(U("method"), U("flickr.test.echo"));
builder.append_query(U("name"), U("value"));
auto path_query_fragment = builder.to_string();
```

The following lines show the initial code that makes the request and returns the resulting `task` instance representing the asynchronous operation. The `request` method indicates that it wants to make an HTTP `GET` request `(methods::GET)` and the query fragment that has to be added `(path_query_fragment)`. Notice that the `then` method adds a continuation `task` to the `task` that executes the request. This way, the continuation task receives the `http_response` instance `(response)` as a parameter after the request is processed. Thus, the code after the `then` continuation is going to be executed in a new task and will check the results of the request. The code is easy to read, as if you were reading synchronous code. It is easy to see that the code block that appears after the `then` continuation is going to be executed after the request task finishes. You don't have to work with complicated futures, you can chain many tasks using the `then` continuation and adding the necessary code to be executed.

```
return client.request(methods::GET, path_query_fragment).
then([](http_response response)
{
…
});
```

In this case, the code defined in the `then` continuation displays the status code returned by the server, the headers content type and length. Everything is easy to access by using the `http_response` received as a parameter `(response)` to the continuation task.

```
std::wostringstream stream;
std::wostringstream stream;
stream << L"Server returned returned status code "
    << response.status_code() << L'.' << std::endl;
std::wcout << stream.str();

stream.str(std::wstring());
stream << L"Content type: " << response.headers().content_
    type() << std::endl;
stream << L"Content length: " << response.headers().content_
    length() << L"bytes" << std::endl;
std::wcout << stream.str();
```

Here, the simple echo call to the Flickr API returns the following XML response. The example just wants to introduce how to build a simple HTTP `GET` request with parameters and process some results with the C++ REST SDK.

```
<rsp stat="fail">
  <err code="100" msg="Invalid API Key (Key has invalid format)"/>
</rsp>
```

The code that reads the response body uses a stream to retrieve the data from the incoming request. In this case, to keep things simple, the code calls the `read_to_end` method and then chains a call to the `get` method in order to wait for the results within the task in which the code is executing.

The `read_to_end` method reads until reaching the end of the stream and returns a `task<size_t>` that contains the two files the number of characters read. Thus, you can call the `read_to_end` method with an asynchronous execution and chain another task to process the read results with the then continuation. In this case, the `get` method waits for the task to finish and returns the result that the task produced. Notice that when a task is canceled, a call to the `get` method will throw a `task_canceled` exception. You usually won't want to use `get` if you aren't within another task (so as to avoid blocking the UI).

```
auto bodyStream = response.body();
streams::stringstreambuf sbuffer;
auto& target = sbuffer.collection();

bodyStream.read_to_end(sbuffer).get();

stream.str(std::wstring());
stream << L"Response body: " << target.c_str();
std::wcout << stream.str();
```

As you can see from this example, executing a simple HTTP `GET` request with C++ 11 and the C++ REST SDK is fairly straightforward. The `then` continuation makes it simple to understand how asynchronous calls are chained and the different helpers included in the C++ REST SDK reduce the necessary boilerplate code to the minimum.

Gastón Hillar is a frequent contributor to Dr. Dobb's.

Comment